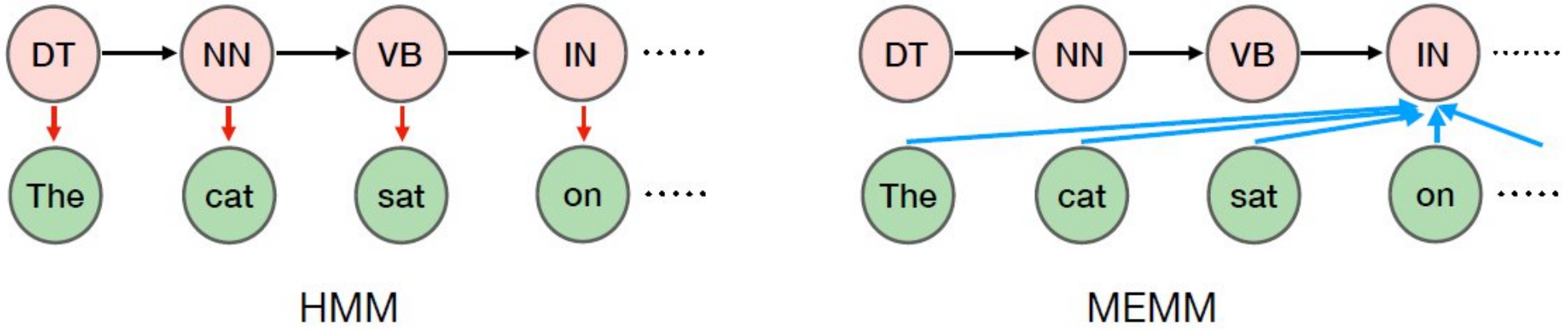
RNN & LSTM

Tran Hong Viet

UET, VNU Ha Noi

How can we model sequences using **neural networks**?



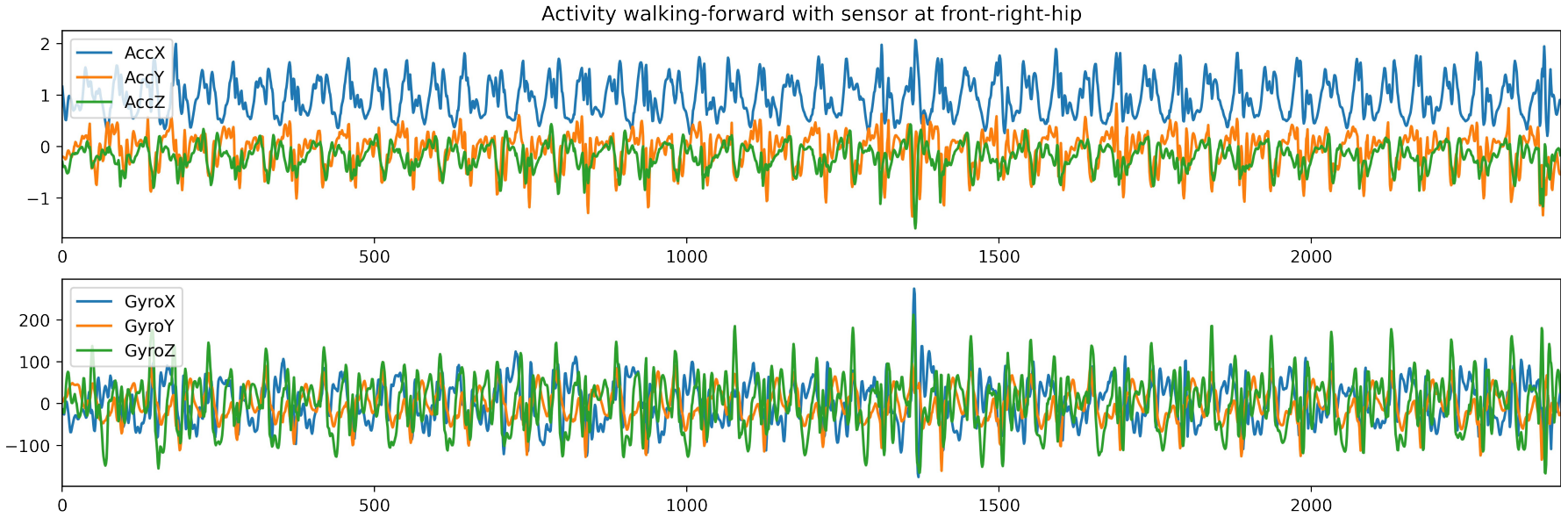
- Architecture of neural networks ? = A class of neural networks used to model sequences, allowing to handle **variable length inputs**
- Very crucial in NLP problems (different from images) because sentences/paragraphs are variable-length, sequential inputs

Content

- Recurrent Neural Network
- The vanishing/exploding gradient problem
- LSTM
- Applications for LSTM

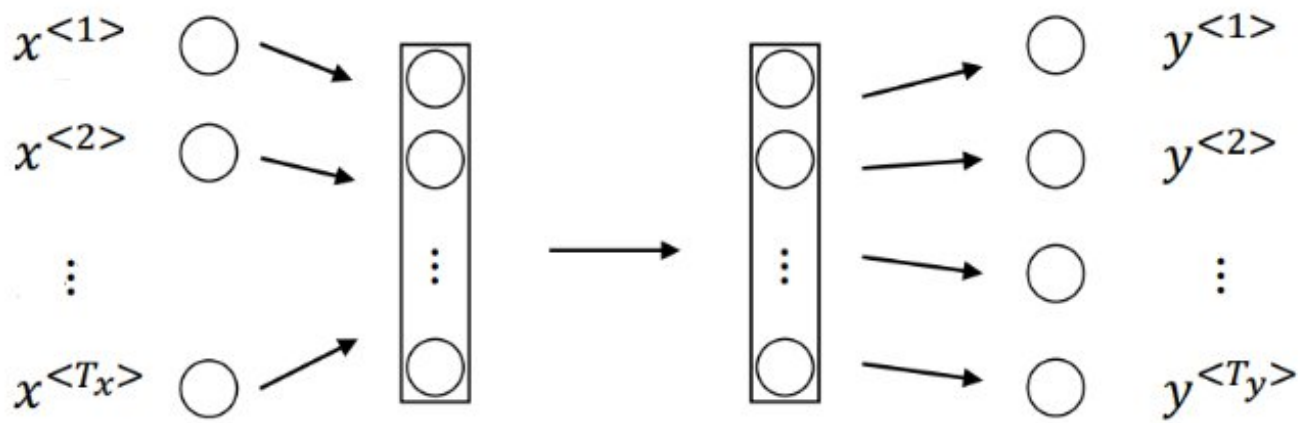
Sequence Data

- Time Series Data
- Natural Language



Data Source: https://dl.acm.org/doi/10.1145/2370216.2370438

Why not standard NN?



Problems:

- Inputs, outputs can be different lengths in different examples.
- Doesn't share features learned across different positions of text.

What is RNN?

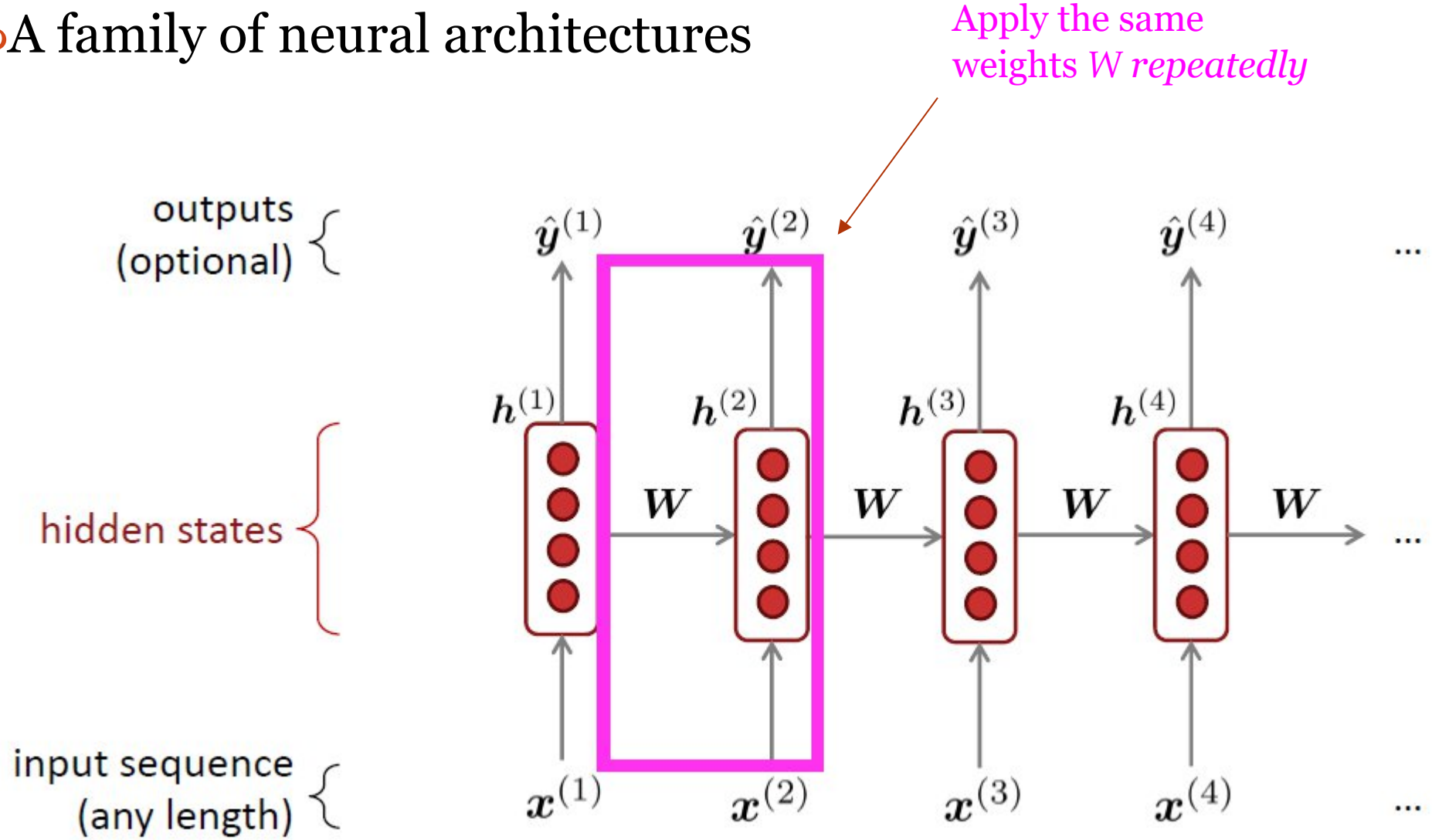
- We consider a class of recurrent networks referred to as **Elman Networks (Elman, 1990)**.
- A **recurrent neural network (RNN)** is a type of artificial neural network which used for sequential data or time series data.

Application:

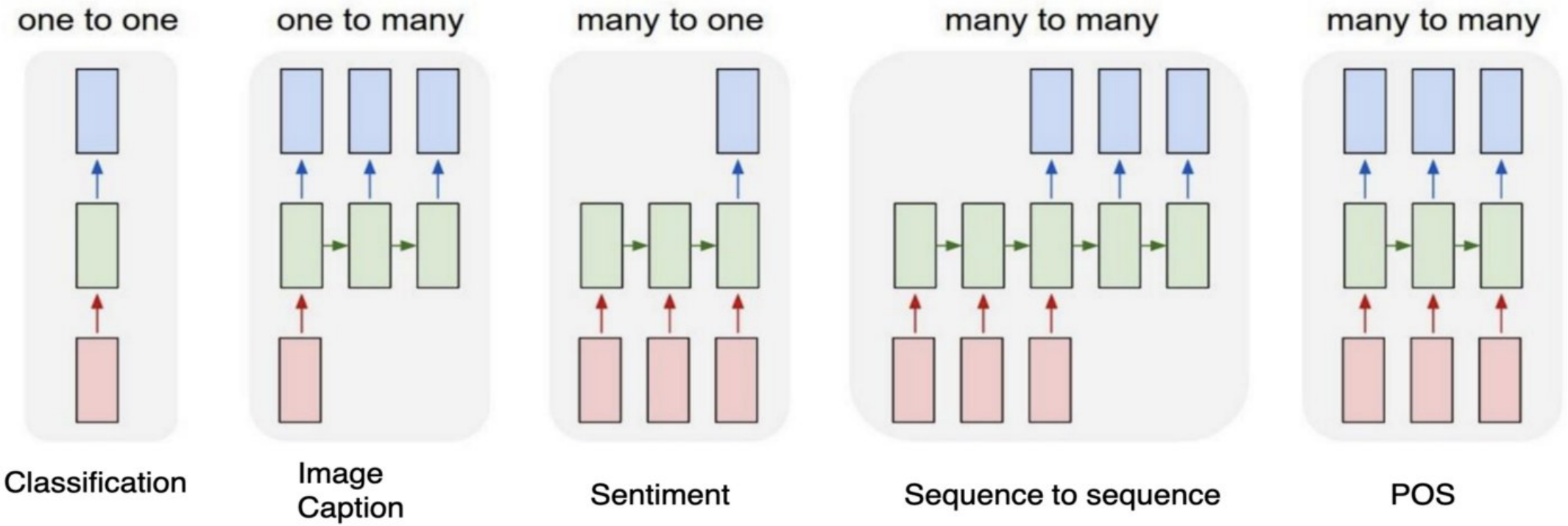
+ Language translation.
+ Natural language processing (NLP).
+ Speech recognition.
+ Image captioning.

Recurrent Neural Networks (RNN)

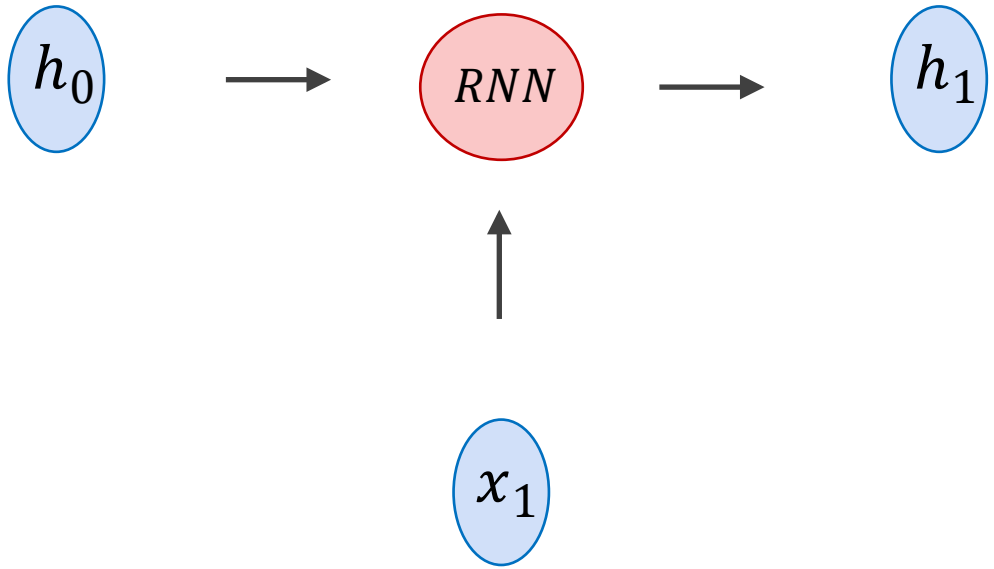
- A family of neural architectures



Types of RNN

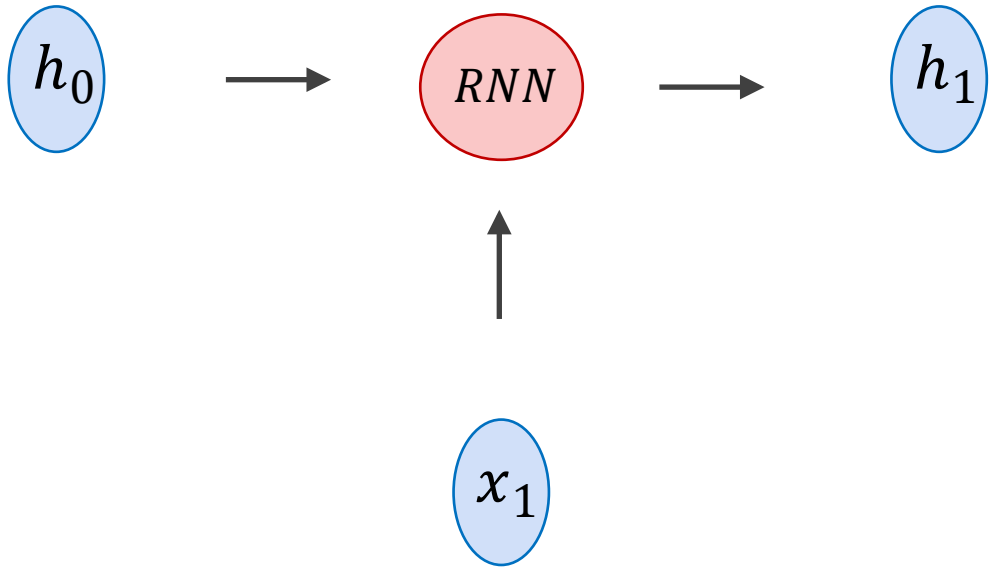

one to one
one to many
many to one
many to many
many to many
Classification
Image Caption
Sentiment
Sequence to sequence
POS

Recurrent Neural Network Cell

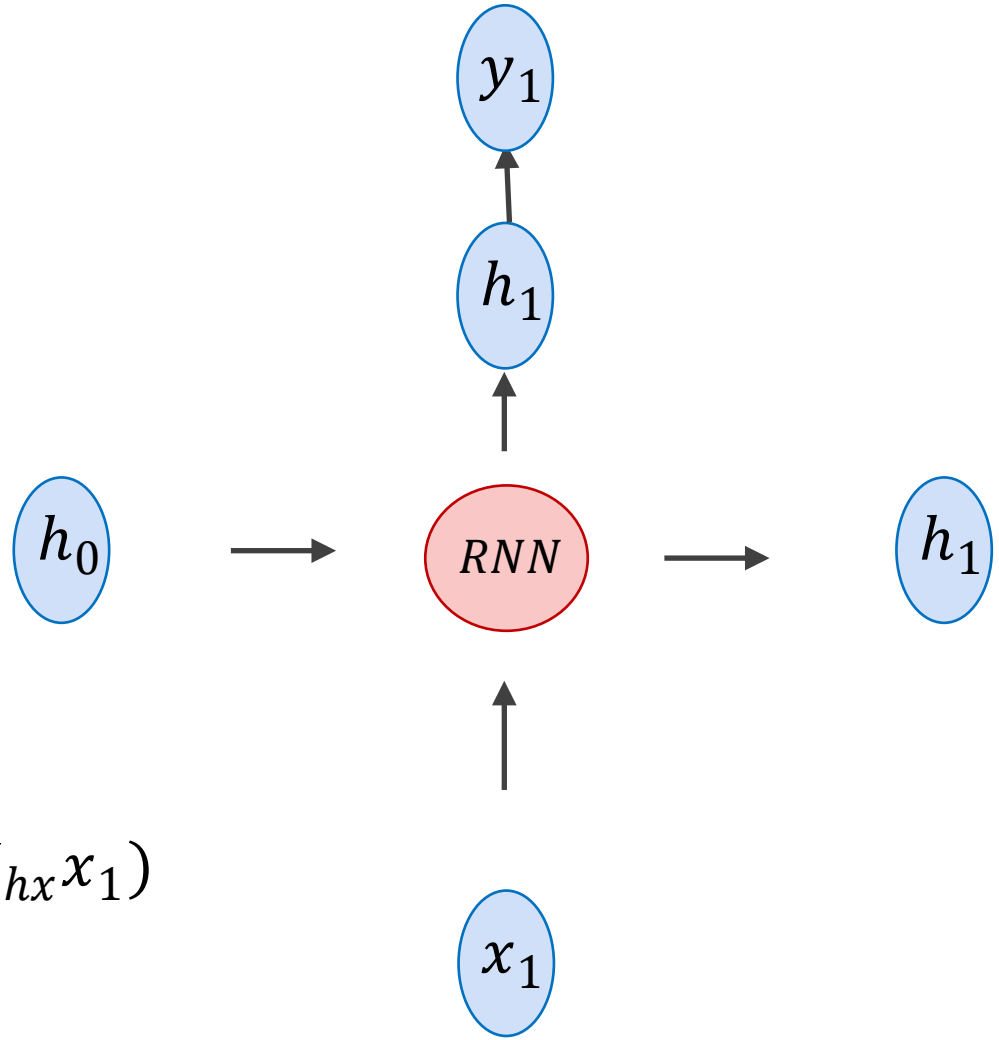


Recurrent Neural Network Cell

$$h_1 = \tanh\left(W_{hh}h_0 + W_{hx}x_1\right)$$



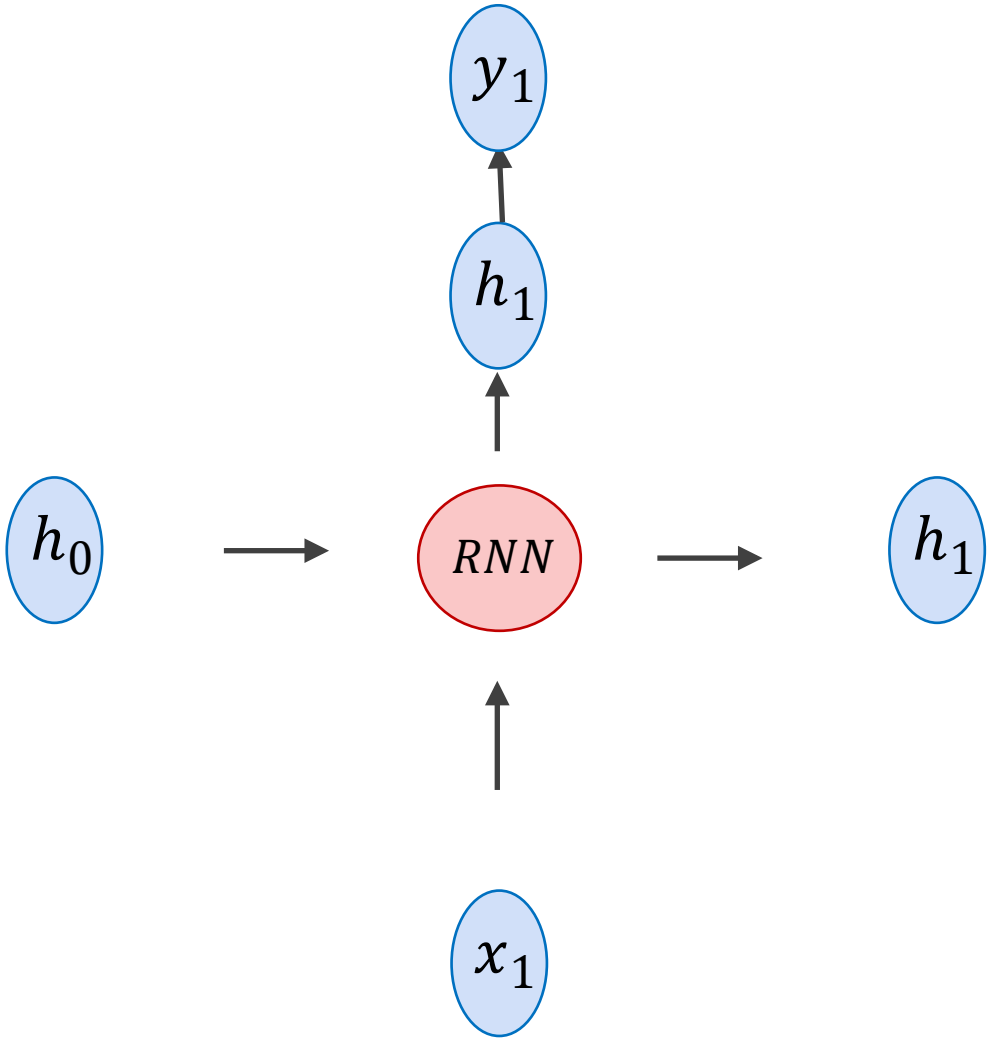
Recurrent Neural Network Cell



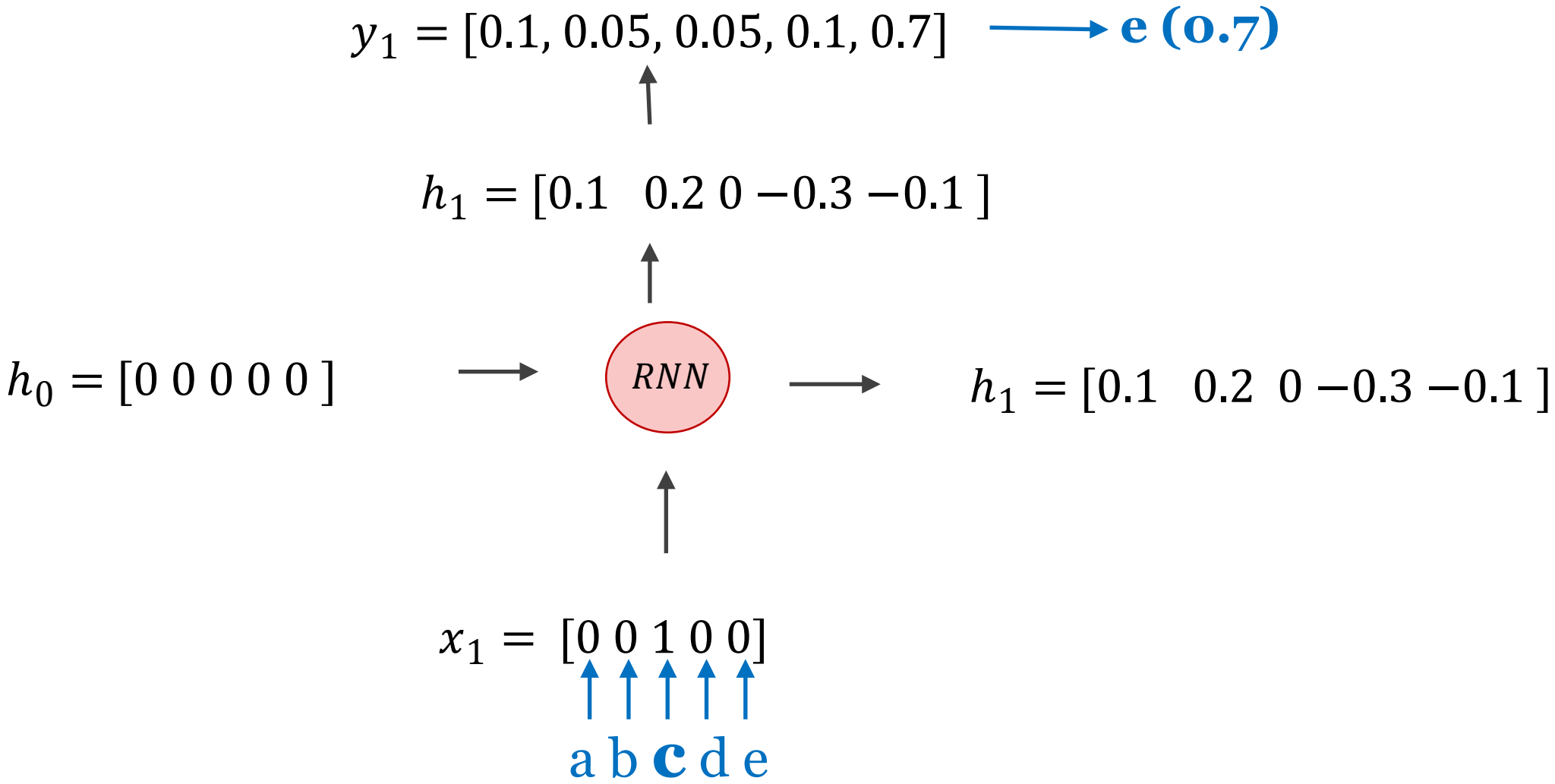
$$h_1 = \tanh\ (W_{hh}h_0 + W_{hx}x_1)$$

$$y_1 = \text{softmax}\ (W_{hy}h_1)$$

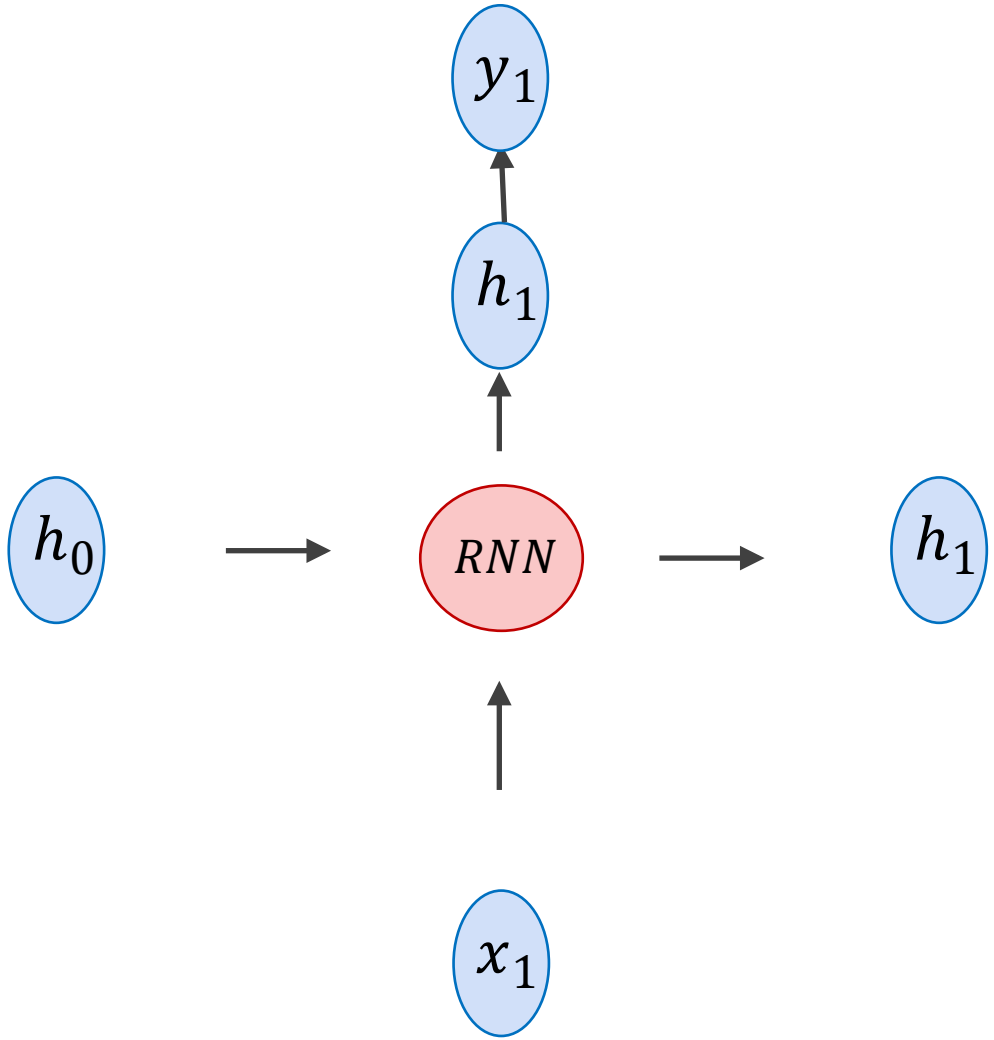
Recurrent Neural Network Cell



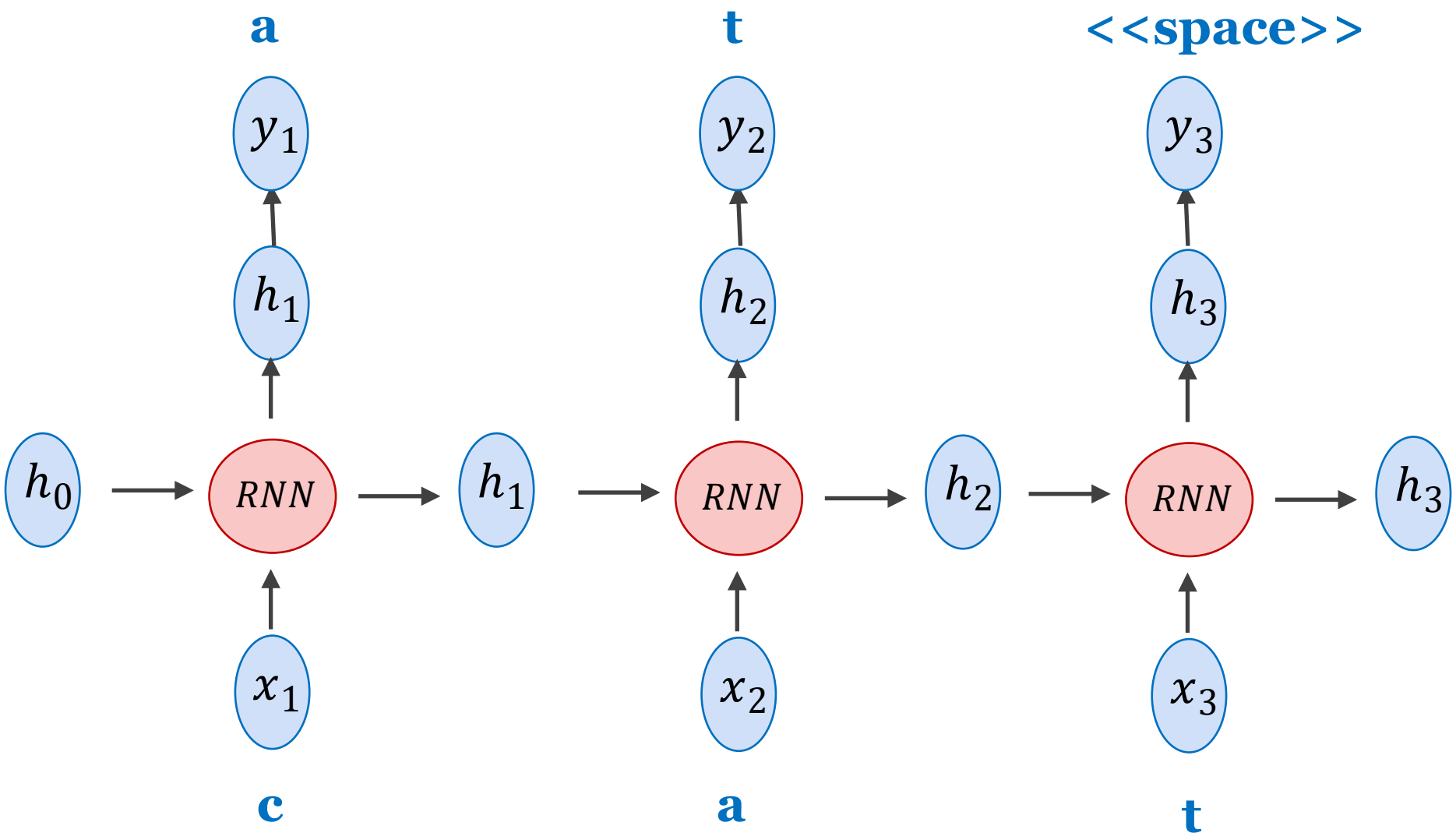
Recurrent Neural Network Cell



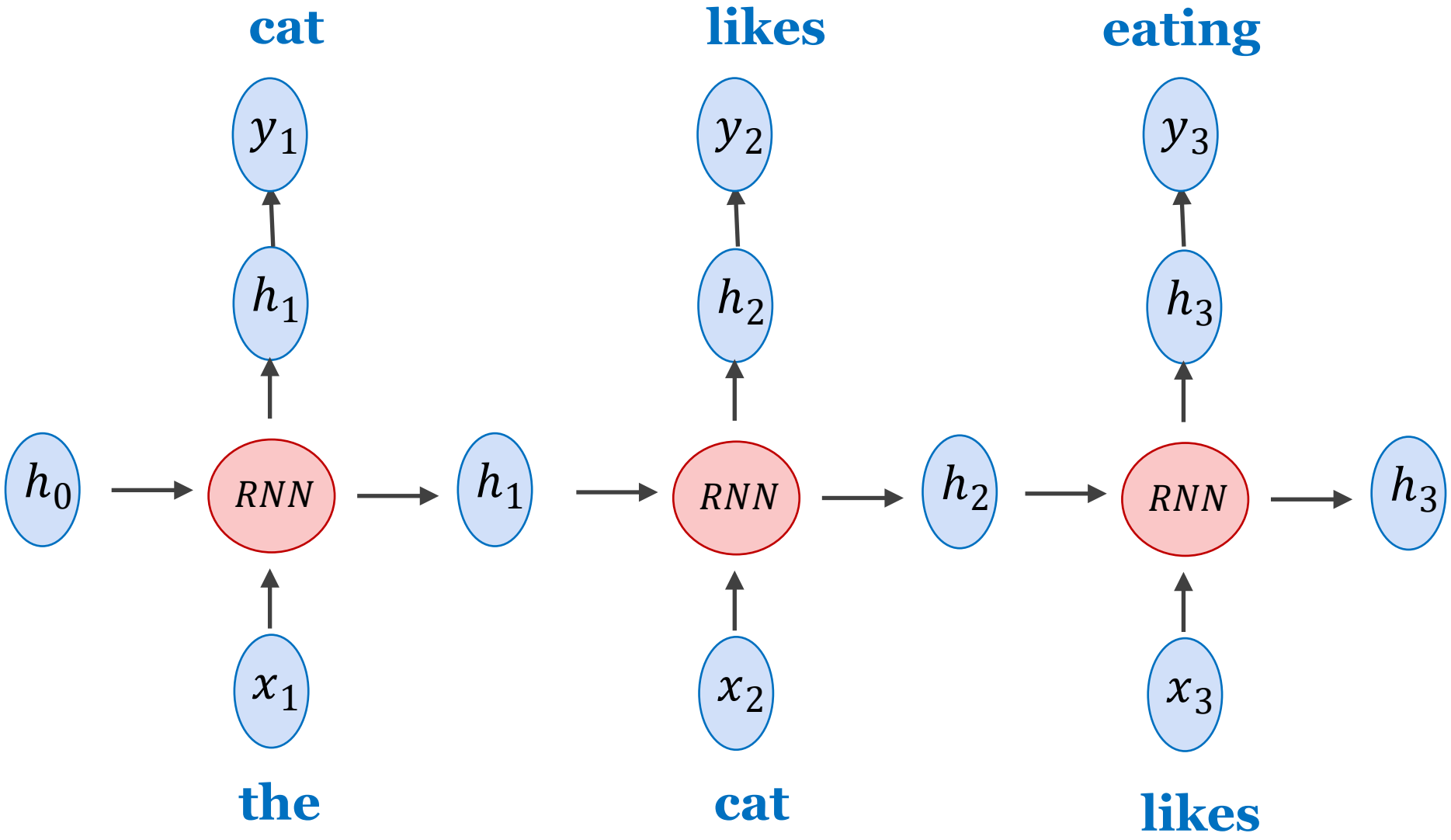
Recurrent Neural Network Cell



(Unrolled) Recurrent Neural Network

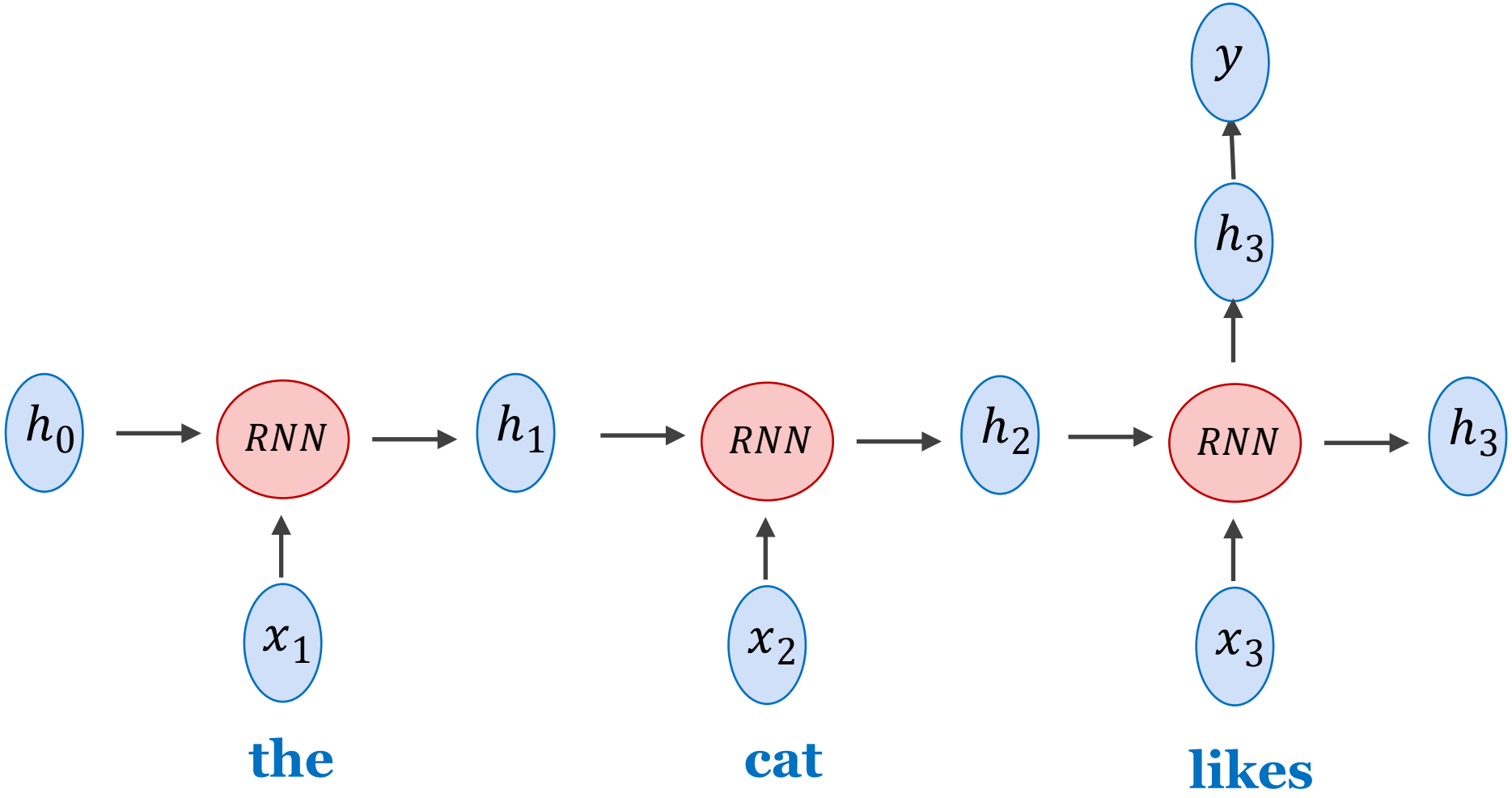


(Unrolled) Recurrent Neural Network

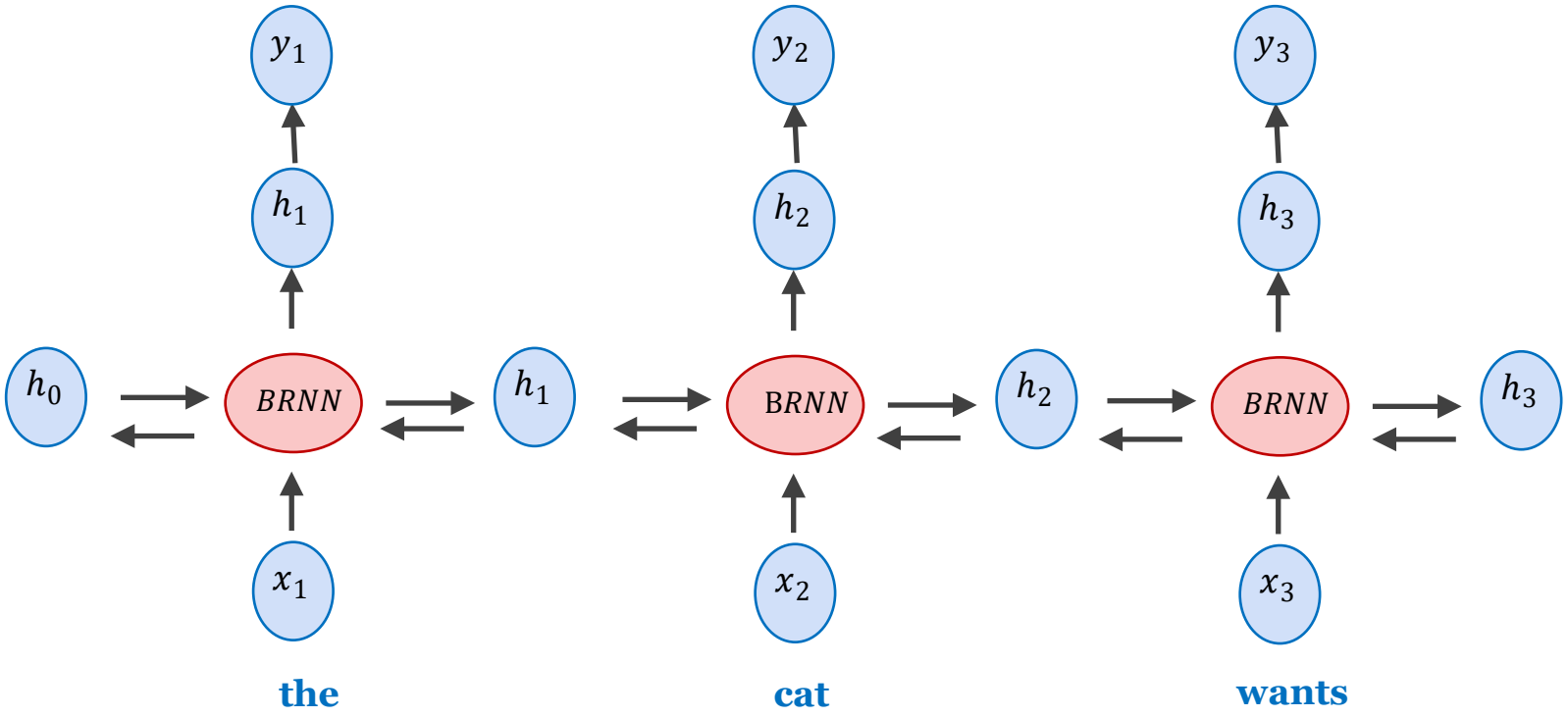


(Unrolled) Recurrent Neural Network

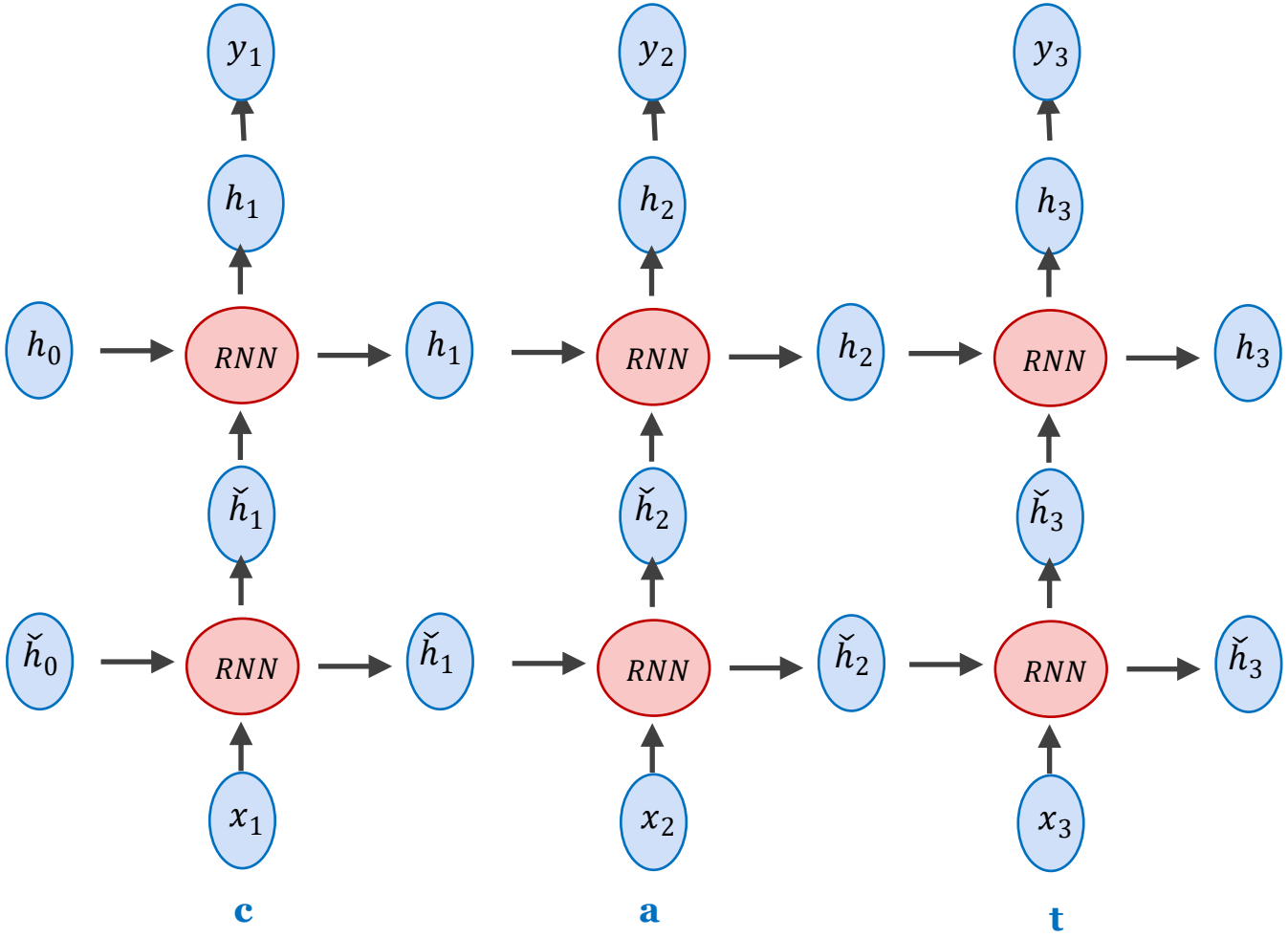
positive / negative sentiment ratin



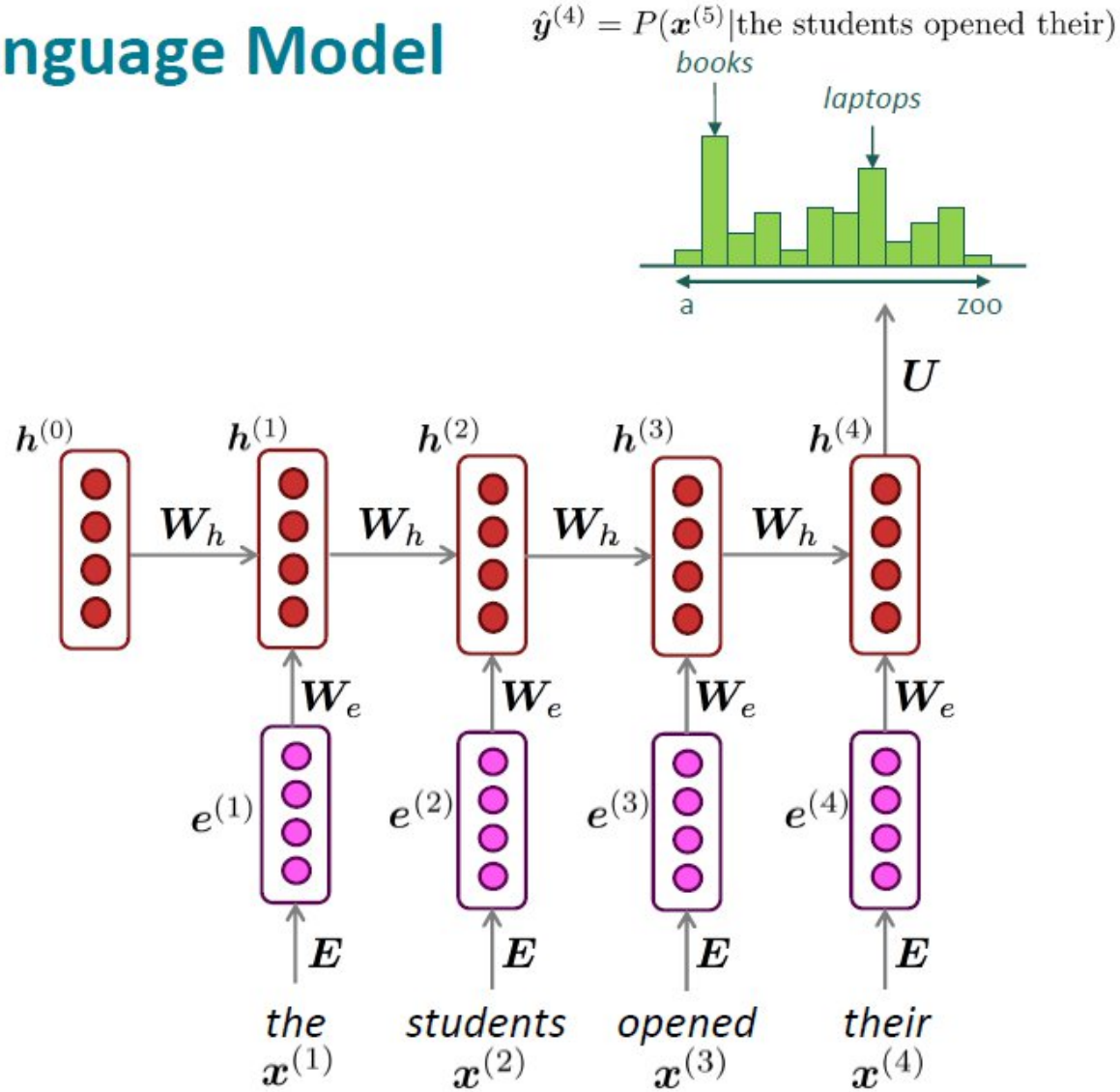
Bidirectional Recurrent Neural Network



Stacked Recurrent Neural Network



A Simple RNN Language Model



output distribution

$$\hat{y}^{(t)} = \text{softmax}\left(Uh^{(t)} + b_2\right) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma\left(W_h h^{(t-1)} + W_e e^{(t)} + b_1\right)$$

$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = Ex^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$

Note: *this input sequence could be much longer now!*

Ac

Training an RNN Language Model

- Get a big corpus of text which is a sequence of words $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{\boldsymbol{y}}^{(t)}$ for *every step t*.
  - i.e., predict probability dist of *every word*, given words so far

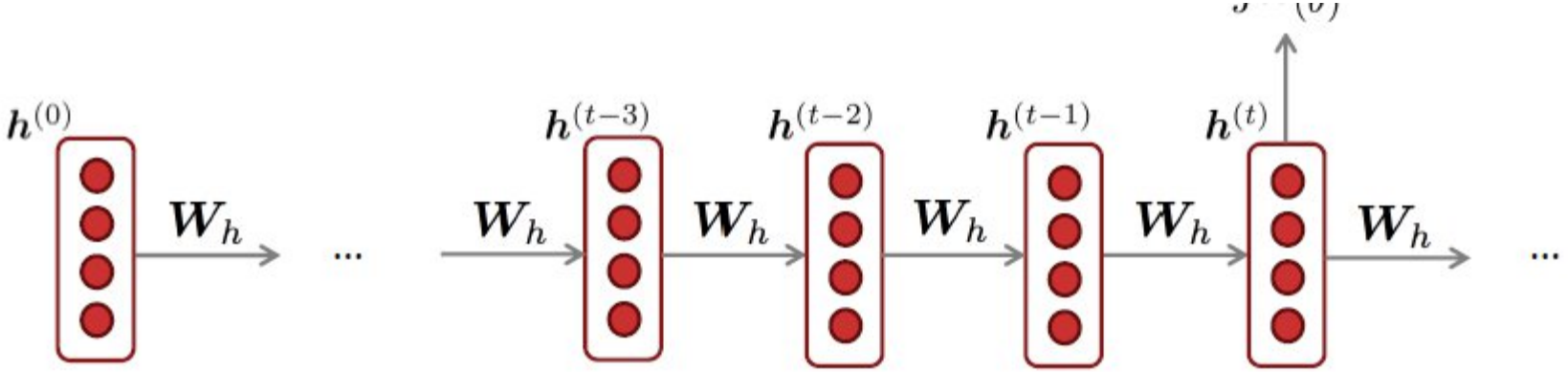
- Loss function on step *t* is cross-entropy between predicted probability distribution $\hat{\boldsymbol{y}}^{(t)}$, and the true next word $\boldsymbol{y}^{(t)}$ (one-hot for $\boldsymbol{x}^{(t+1)}$):

$$J^{(t)}(\theta) = CE(\boldsymbol{y}^{(t)}, \hat{\boldsymbol{y}}^{(t)}) = -\sum_{w \in V} \boldsymbol{y}_w^{(t)} \log \hat{\boldsymbol{y}}_w^{(t)} = -\log \hat{\boldsymbol{y}}_{\boldsymbol{x}_{t+1}}^{(t)}$$

- Average this to get overall loss for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^{T} -\log \hat{\boldsymbol{y}}_{\boldsymbol{x}_{t+1}}^{(t)}$$

Backpropagation for RNNs



Question: What's the derivative of $J^{(t)}(\theta)$ w.r.t. the repeated weight matrix $\boldsymbol{W}_h$?

Answer: $$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\Big|_{(i)}$$

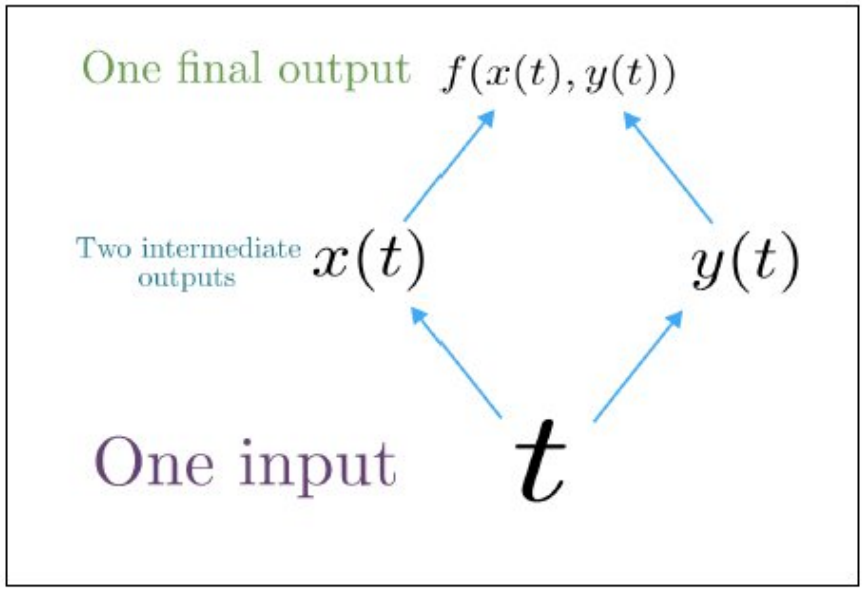
"The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears"

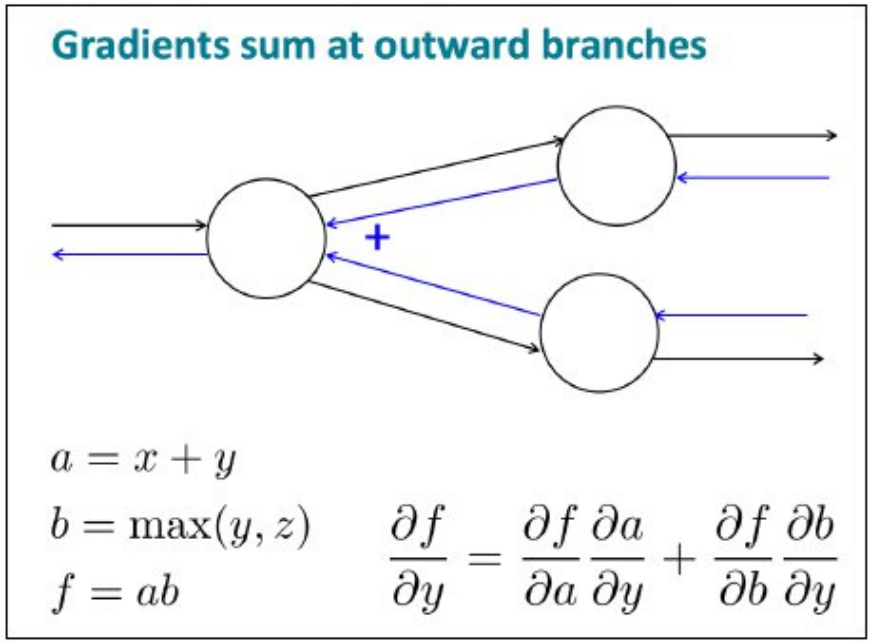
Why?

Multivariable Chain Rule

- Given a multivariable function $f(x, y)$, and two single variable functions $x(t)$ and $y(t)$, here's what the multivariable chain rule says:

$$\underbrace{\frac{d}{dt} f(x(t), y(t))}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}$$

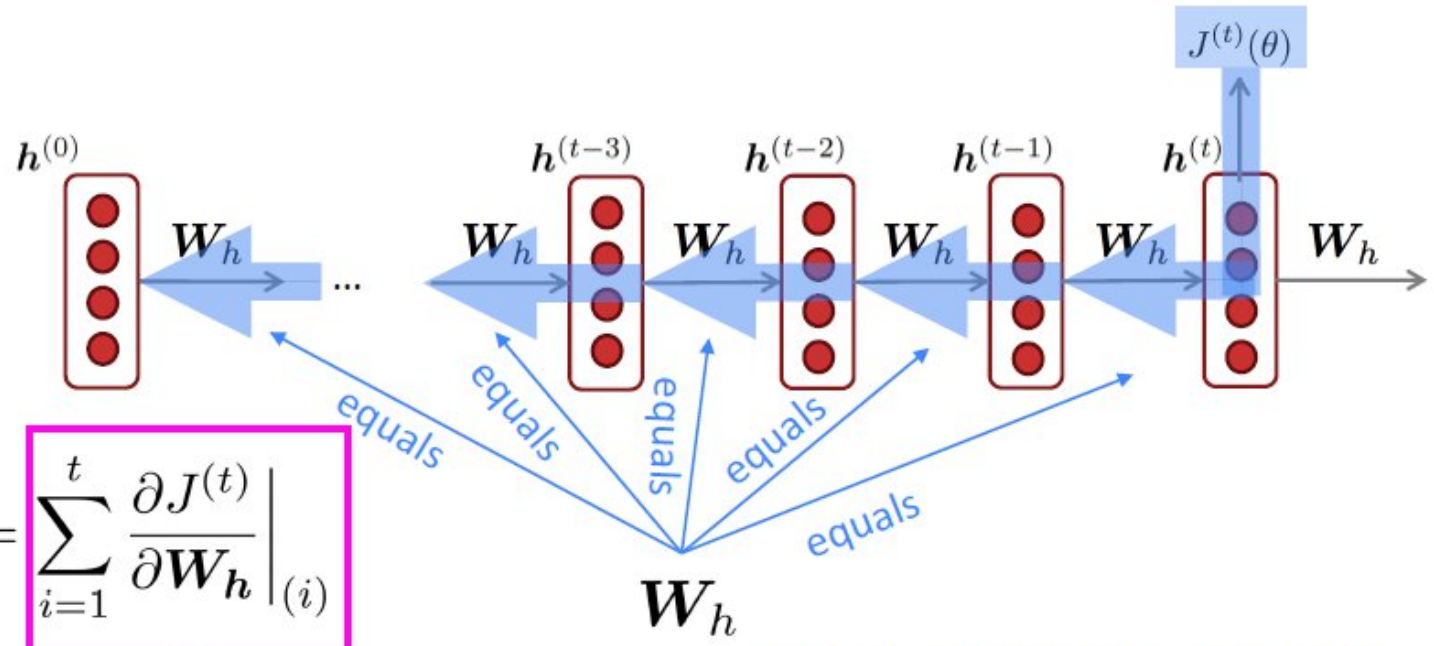




Source: https://www.khanacademy.org/math/multivariable-calculus/multivariable-derivatives/differentiating-vector-valued-functions/a/multivariable-chain-rule-simple-version

Backpropagation for RNNs

In practice, often "truncated" after ~20 timesteps for training efficiency reasons



$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)}$$

Question: How do we calculate this?

Answer: Backpropagate over timesteps $i = t, \ldots, 0$, summing gradients as you go. This algorithm is called **"backpropagation through time"** [Werbos, P.G., 1988, *Neural Networks* **1**, and others]

Apply the multivariable chain rule:

$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)} \frac{\partial \boldsymbol{W_h}|_{(i)}}{\partial \boldsymbol{W_h}}$$

$= 1$

$$= \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)}$$

Backpropagation through time

$$\mathbf{h}_1 = g(\mathbf{W}\mathbf{h}_0 + \mathbf{U}\mathbf{x}_1 + \mathbf{b})$$
$$\mathbf{h}_2 = g(\mathbf{W}\mathbf{h}_1 + \mathbf{U}\mathbf{x}_2 + \mathbf{b})$$
$$\mathbf{h}_3 = g(\mathbf{W}\mathbf{h}_2 + \mathbf{U}\mathbf{x}_3 + \mathbf{b}) \qquad \hat{\mathbf{y}}_3 = \text{softmax}(\mathbf{W}_o\mathbf{h}_3)$$

$$L_3 = -\log \hat{\mathbf{y}}_3(w_4)$$

First, compute gradient with respect to hidden vector of last time step: $\frac{\partial L_3}{\partial \mathbf{h}_3}$

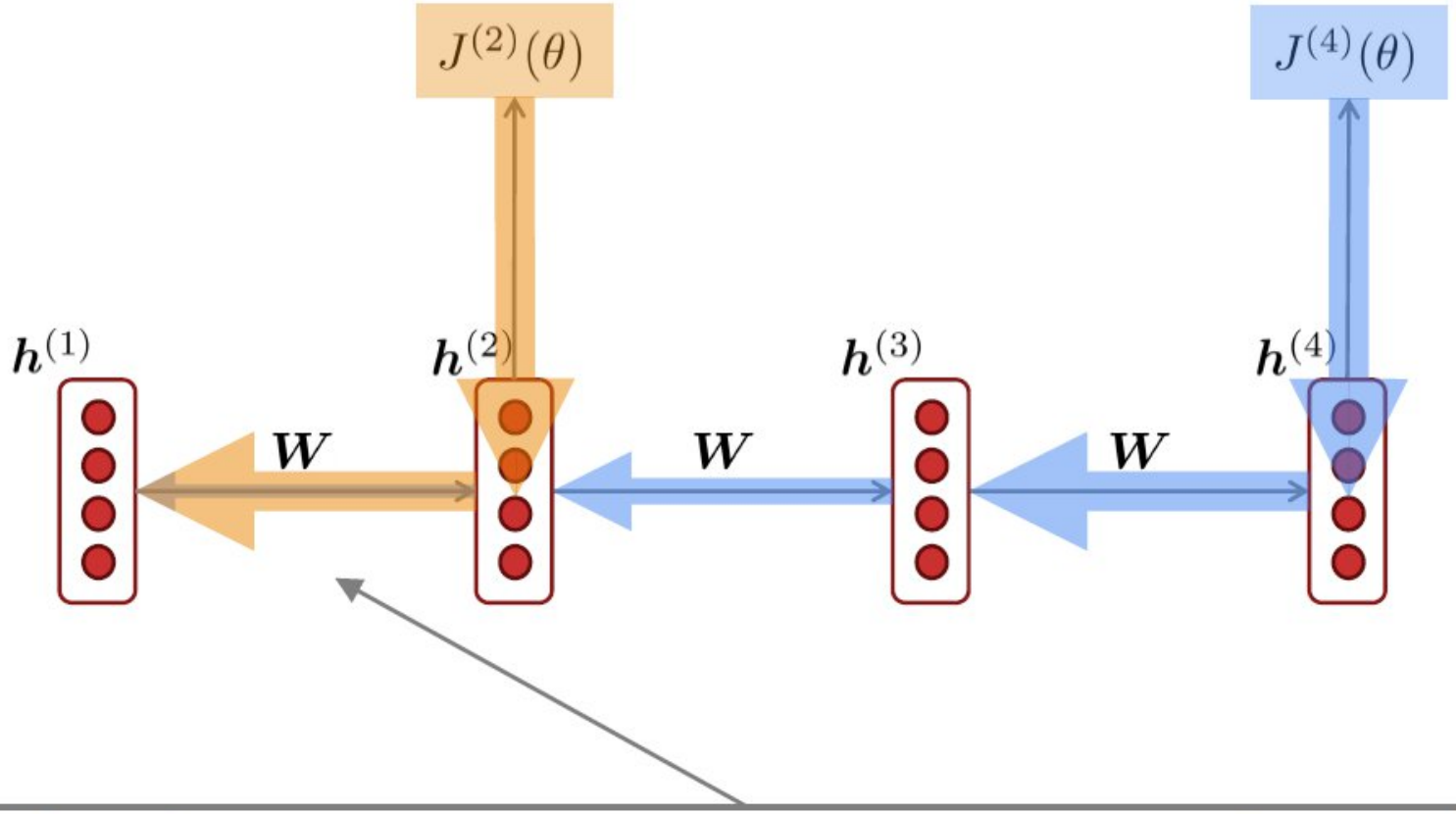
$$\frac{\partial L_3}{\partial \mathbf{W}} = \frac{\partial L_3}{\partial \mathbf{h}_3}\frac{\partial \mathbf{h}_3}{\partial \mathbf{W}} + \frac{\partial L_3}{\partial \mathbf{h}_3}\frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2}\frac{\partial \mathbf{h}_2}{\partial \mathbf{W}} + \frac{\partial L_3}{\partial \mathbf{h}_3}\frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2}\frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1}\frac{\partial \mathbf{h}_1}{\partial \mathbf{W}}$$

More generally,

$$\boxed{\frac{\partial L}{\partial \mathbf{W}} = -\frac{1}{n}\sum_{t=1}^{n}\sum_{k=1}^{t}\frac{\partial L_t}{\partial \mathbf{h}_t}\left(\prod_{j=k+1}^{t}\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}\right)\frac{\partial \mathbf{h}_k}{\partial \mathbf{W}}}$$

If *k* and *t* are far away, the gradients can grow/shrink exponentially (called **the gradient exploding or gradient vanishing problem**)

Why is vanishing gradient a problem?



Gradient signal from far away is lost because it's much smaller than gradient signal from close-by.

So, model weights are updated only with respect to near effects, not long-term effects.

The vanishing gradient problem for language models

- **Example (RNN-LM task):**

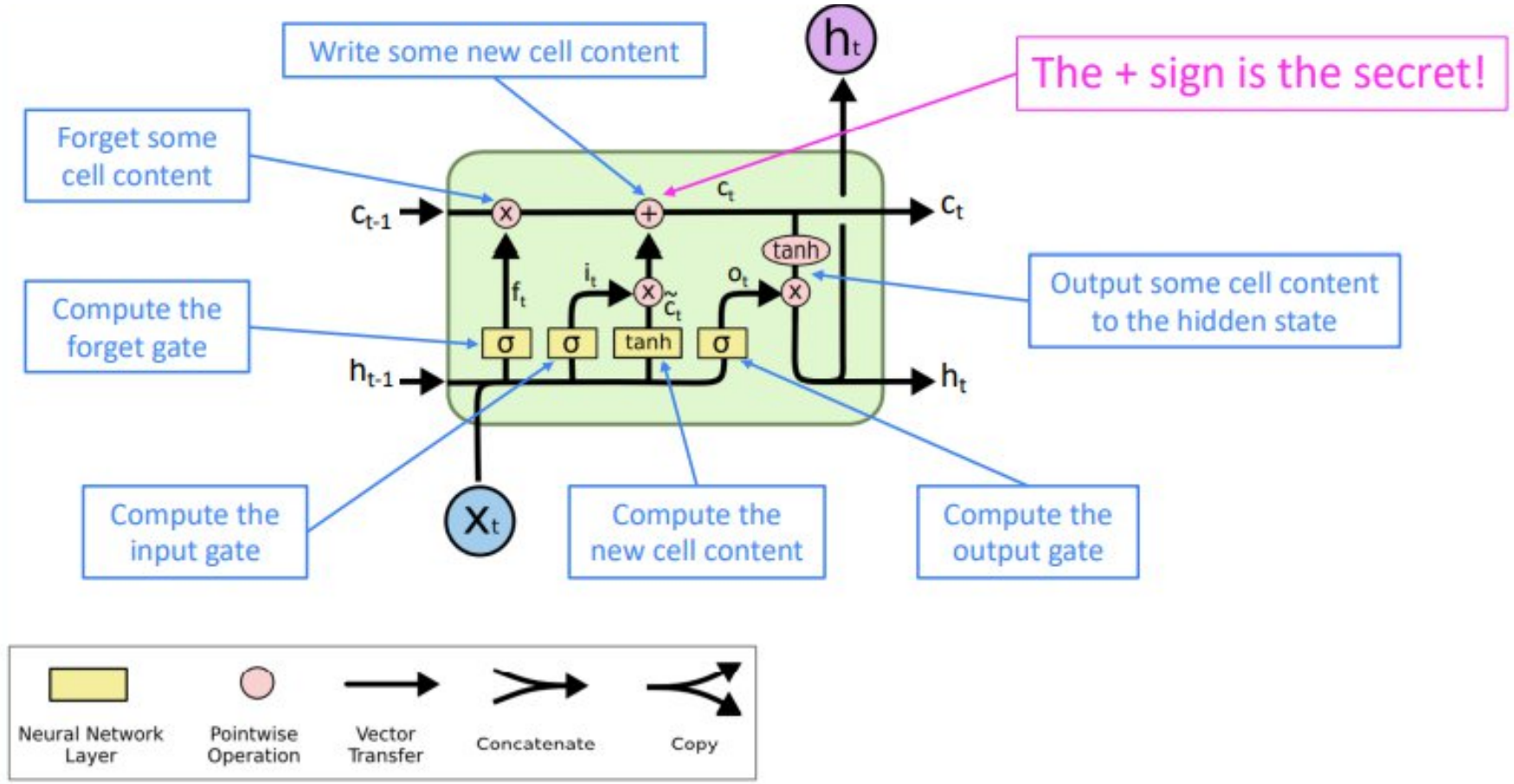
Jane walked into the room. John walked in too. It was late in the day. Jane said hi to _____

- To learn from this training example, the RNN-LM needs to model the dependency between *"John"* on the 7th step and the target word *"John"* at the end.
- But if the gradient is small, the model **can't learn this dependency**
  - So, the model is **unable to predict similar long-distance dependencies** at test time

Vanishing/Exploding Solutions

- **Vanishing Gradient:**
  - Gating mechanism (LSTM, GRU)
  - Attention mechanism (Transformer)
  - Adding skip connection through time (Residual connection)
  - Better Initialization

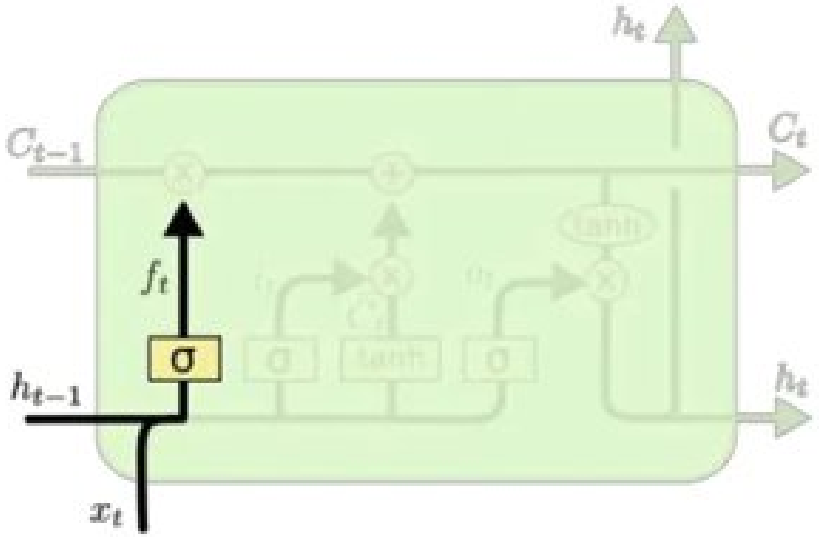
Long-Short Term Memory (LSTM) - 1997



LSTM, Hochreite & Schmidhuber, 1997, https://deeplearning.cs.cmu.edu/F23/document/readings/LSTM.pdf

Architecture of LSTM cell

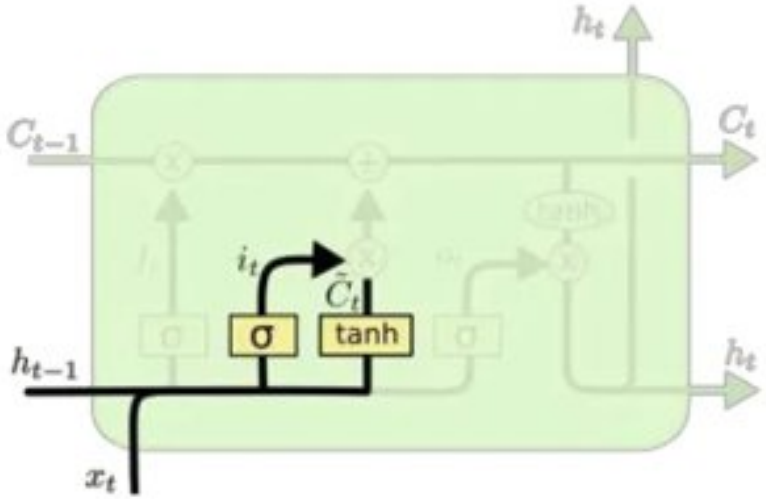
- **Forget information:**
  - Decide what information throw away from the cell state
  - **Forget gate layer:**
    - Output a number between 0 and 1



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Architecture of LSTM cell

- **Add new information:**
  - Decide what new information store in the cell state
  - **Input gate layer:**
    - Decides which values we'll update
  - **Tanh layer:**
    - creates a vector of new candidate values, $\widetilde{C}_t$

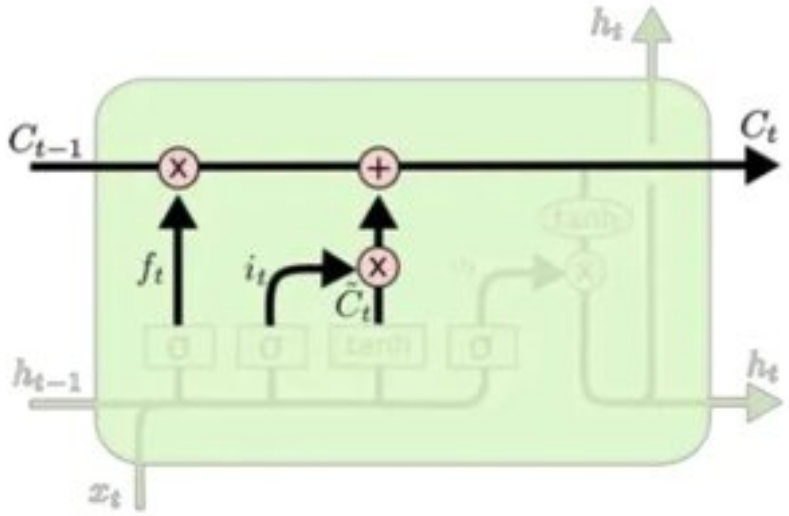


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Architecture of LSTM cell

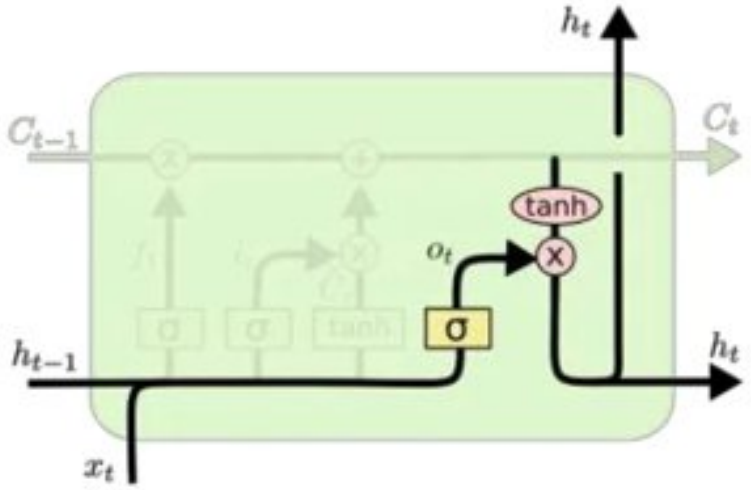
- **Update cell state:**
  - Forgetting the things we decided to forget earlier: $f_t * C_{t-1}$
  - Adding information we decide to add: $i_t * \tilde{C}_t$



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Architecture of LSTM cell

- **Create output:**
  - Decide what we're going to output
  - **Output gate layer:**
    - Decides what parts of the cell state we're going to output
  - **Tanh layer:**
    - Push the values between -1 and +1



$$o_t = \sigma\left(W_o\ [h_{t-1}, x_t]\ +\ b_o\right)$$

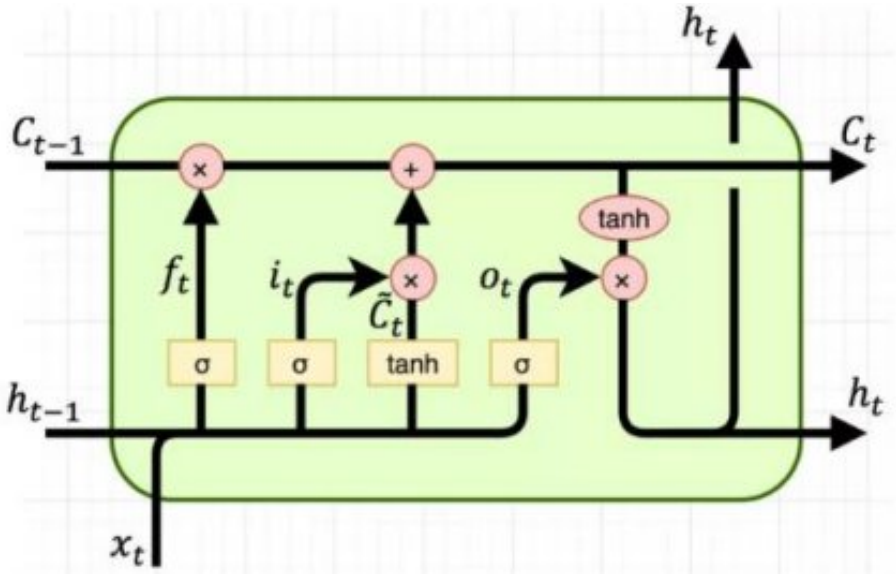
$$h_t = o_t * \tanh\left(C_t\right)$$

↓

Shadow state/ Short memory

Architecture of LSTM cell

- **Conclusion**:
  - Step 1: Forget gate layer.
  - Step 2: Input gate layer.
  - Step 3: Combine step 1 & 2.
  - Step 4: Output the cell state.

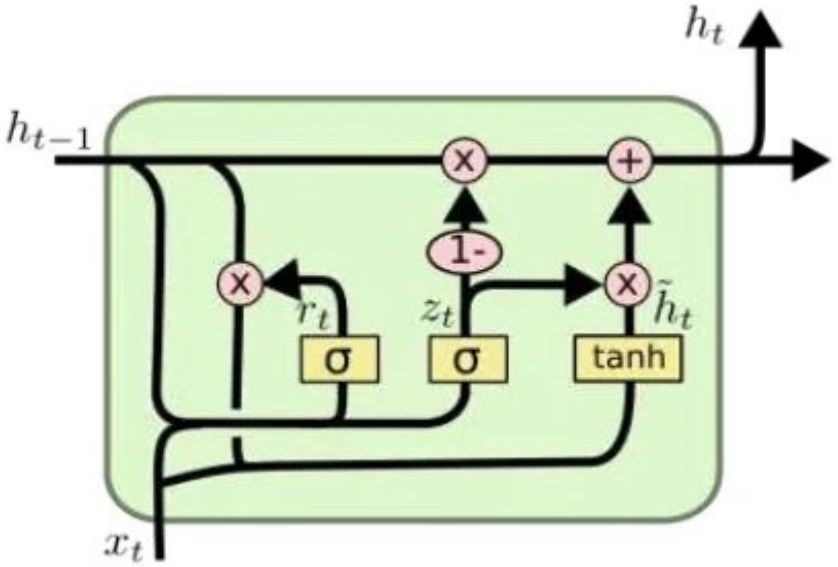


How does LSTM can solve vanishing gradient

- The LSTM architecture makes it **easier** for the RNN to **preserve** information **over many timesteps**.
- LSTM ***doesn't guarantee*** that there is **no vanishing/ exploding** gradient.
- LSTM provides an **easier way** for the model to learn **long-distance dependencies**.

LSTM Variations (GRU)

- **Gated Recurrent Unit (GRU) (** Kyunghyun Cho et al., 2014)
  - Combine the forget and input layer into a single “update gate”
  - Merge the cell state and the hidden state
  - Simpler.



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Compare LSTM vs. GRU

- **GRUs train faster** and perform better than LSTMs on **less training data** if you are doing language modeling (not sure about other tasks).
- **GRUs are simpler** and thus easier to modify, for example adding new gates in case of additional input to the network. It's just less code in general.
- **LSTMs** should in theory **remember longer sequences** than GRUs and outperform them in tasks requiring modeling long-distance relations.

Successful Applications of LSTMs

- Speech recognition: Language and acoustic modeling
- Sequence labeling
  - POS Tagging
    https://www.aclweb.org/aclwiki/index.php?title=POS_Tagging_(State_of_the_art)
  - NER
  - Phrase Chunking
- Neural syntactic and semantic parsing
- Image captioning: CNN output vector to sequence
- Sequence to Sequence
  - Machine Translation (Sustkever, Vinyals, & Le, 2014)
  - Video Captioning (input sequence of CNN frame outputs)

Summary

- Recurrent Neural Network is one of the best deep NLP model families
- Most important and powerful RNN extensions with LSTMs and GRUs

Homework

- RNN & LSTM for **sentiment analysis**
- Corpus of IMDB: The IMDB movie reviews dataset is a set of 50,000 reviews, half of which are positive and the other half negative
- Compare the results with previous methods (SVM, Logistic Regression)

References

- Speech and Language Processing (3rd ed. draft), chapter 9
- Slide of Stanford NLP course and other documents

Question and Discussion!